STAGE 1 · VISUAL NOTES

Image Formation

How light becomes the numbers `cv2.imread` gives you

These are the same notes as the text version, with a figure for every key idea. **Read the picture first, then the words under it.**

> **How to use these notes**
>
> Each lesson opens with its story and key symbols, then the core concepts with figures, the formulas (every symbol explained), the mission, every exercise with your answer and its evaluation, and tips. To revise, look at a figure and explain it aloud *before* reading the text under it.

> **Notation conventions used throughout**
>
> - **Italic letters** (f, Z, λ) are quantities; upright text (SNR, MSE) is a named measure.
> - **Square brackets** $I[m, n]$ mean a discrete pixel grid; **round brackets** $I(x, y)$ mean a continuous function.
> - $\propto$ means "is proportional to"; $\approx$ means "approximately equal"; $|\cdot|$ is absolute value.
> - A few letters are reused in different lessons (for example H is image height in Lesson 1 but object height in Lesson 2). Every formula box says which meaning applies, and the glossary at the end lists all of them.

Overview

Stage 1 follows light from the scene all the way to the array that `cv2.imread` returns. Each lesson explains one box of the pipeline below; the orange tag under a box names the lesson and its key formula.

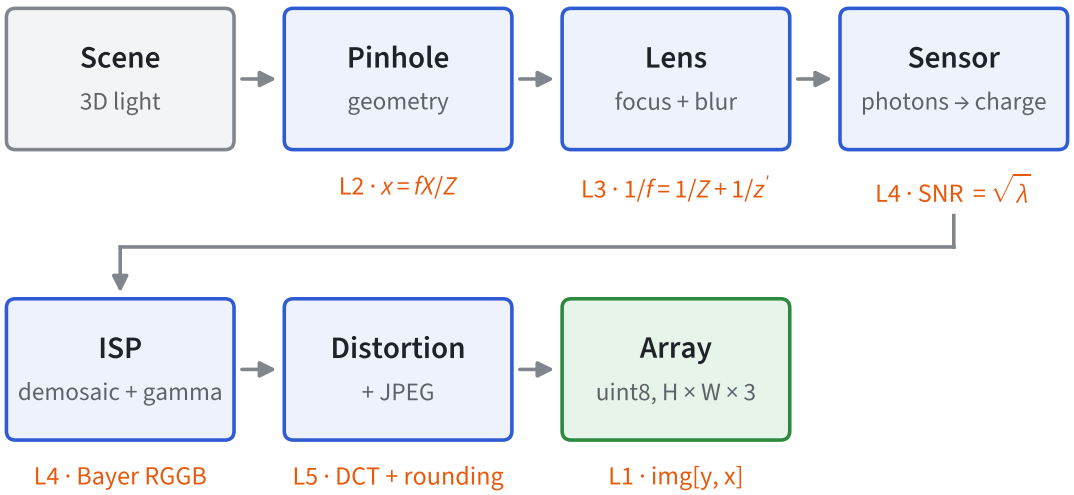


Figure 1. The Stage 1 pipeline. Light leaves the scene, is projected by pinhole geometry, focused (and blurred) by a lens, counted by the sensor, processed by the ISP, warped by lens distortion and compressed by JPEG, and finally arrives as a `uint8` array of shape $H \times W \times 3$.

Table 1. The five lessons at a glance.

Lesson	Core idea	Key formula
1 · Images as numbers	Sampling, quantisation, arrays, data types, colour channels	$Y = 0.299R + 0.587G + 0.114B$
2 · Pinhole camera	Perspective projection, lost depth, dolly zoom	$x = fX/Z$
3 · Lenses	Thin lens, circle of confusion, depth of field, f-number	$1/f = 1/Z + 1/z'$
4 · Sensors	Exposure, shot noise, averaging, Bayer filter and the ISP	$\mathrm{SNR} = \sqrt{\lambda}$
5 · Distortion and JPEG	Radial distortion, backward mapping, DCT and quantisation	$r_d = r(1 + k_1 r^2 + k_2 r^4)$

LESSON 1

Images as numbers

An image is a landscape of light, $I(x, y)$, turned into a grid of integers by two approximations: **sampling** (space) and **quantisation** (brightness).

Story to remember

In 1957 Russell Kirsch scanned a photo of his baby son Walden on the SEAC computer. It was 176 × 176 pixels, 1 bit each (black or white), and it invented the pixel. Kirsch later regretted choosing square pixels, which is why diagonal lines show staircases. The Kirsch edge operator is also named after him.

Key symbols in this lesson

$I(x, y)$	brightness of the continuous scene image at position (x, y)
$I[m, n]$	the stored pixel value at row m, column n
Δ	spacing between samples (the pixel pitch)
$b,\ L$	bits per value, and the number of brightness levels $L = 2^b$
H, W, C	image height (rows), width (columns) and number of channels
R, G, B	red, green and blue channel values of a pixel

Core concepts

1. Image as a function: the landscape picture

Lay a photo flat and raise each point by its brightness. Bright sky becomes a plateau, a dark tree becomes a valley, and the border between them becomes a cliff.

- An **edge** is a cliff: a place where brightness changes fast, so the derivative is large (the orange dashed line).
- **Noise** is small bumps on a flat plain (the 19, 21, 23 wobbles).
- **Blurring** is erosion that turns cliffs into slopes.

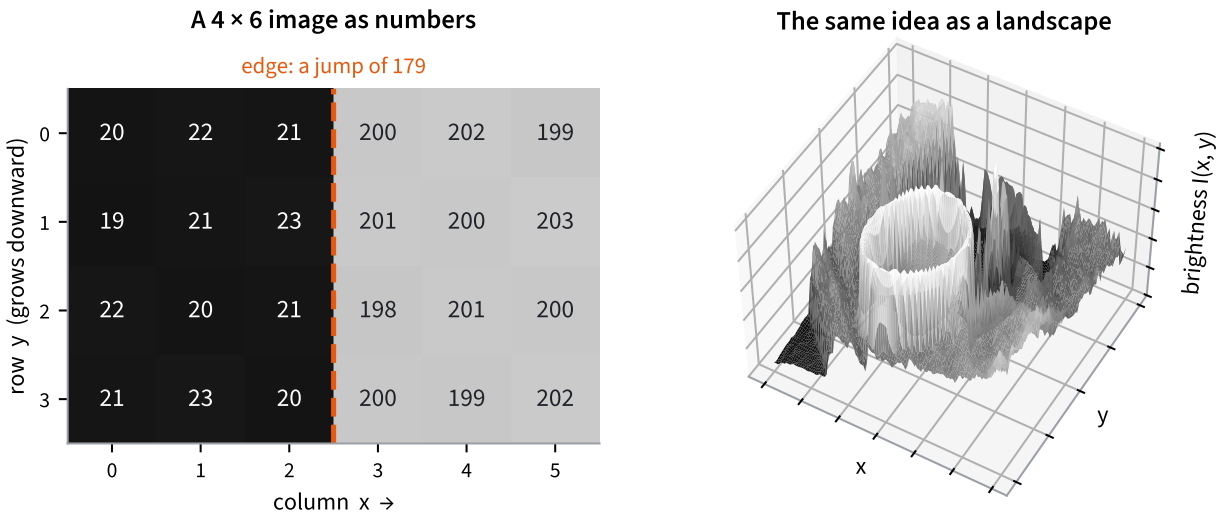


Figure 2. Left: a 4 × 6 image stored as numbers; dark values (≈ 20) meet bright ones (≈ 200) at the orange dashed line. **Right:** a real photo drawn as a landscape, where height is brightness $I(x, y)$.

- **Colour** is three stacked landscapes, one each for R, G and B.

2. Sampling: space becomes a grid

The sensor is a grid of photosites (light buckets). Each reports one number, a pixel. The number of samples is the resolution: 4000 × 3000 = 12 megapixels (MP).

Too coarse a grid does not just lose detail, it **invents false detail**. This is **aliasing**: moiré on a striped shirt, rainbow waves when you photograph a monitor, and the wagon-wheel effect (24 frames per second samples time too slowly).

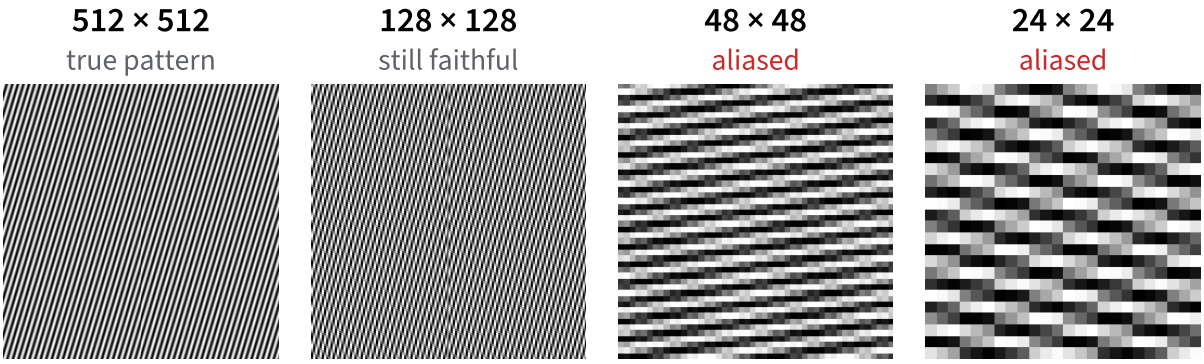


Figure 3. Sampling the same fine stripe pattern on coarser and coarser grids. At 128 × 128 the stripes survive; at 48 × 48 and 24 × 24 the real stripes turn into a few broad, fake waves: aliasing.

3. Quantisation: brightness becomes levels

With b bits per value you get 2^b brightness levels. Too few levels causes **banding**: a smooth sky breaks into stripes.

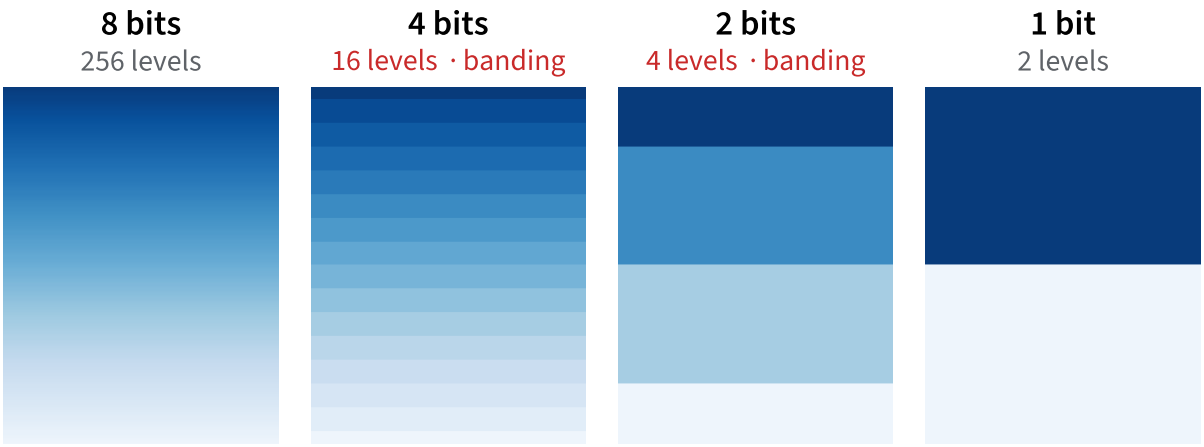


Figure 4. The same smooth sky stored with fewer and fewer bits. At 4 bits you can count 16 bands; at 1 bit you see what Kirsch's 1957 image could store.

Table 2. Common bit depths.

Bits b	Levels 2^b	Where you see it
1	2	Kirsch's 1957 image, fax, QR codes
8	256	Normal photos, JPEGs, screens
10–12	1,024–4,096	HDR video, RAW files
16	65,536	Medical CT/MRI, scientific imaging

4. Storage: an image is an array

A grayscale image has shape (H, W); a colour image has shape $(H, W, 3)$, where H is the number of rows and W the number of columns. The origin $(0, 0)$ is the **top-left**, y increases **downward**, and indexing is `img[row, col]` = `img[y, x]` (CRT beams painted from the top-left, line by line).

A 12 MP colour photo takes 4000 × 3000 × 3 ≈ 36 MB uncompressed.

Table 3. Array layout by library. C is the number of channels.

Library	Shape	Channel order
OpenCV	(H, W, C)	BGR
Matplotlib / PIL	(H, W, C)	RGB
PyTorch	(C, H, W)	RGB

5. Data types: where numbers break

`uint8` holds whole numbers from 0 to 255. When a result goes past 255, NumPy **wraps** around (300 → 44) while OpenCV **saturates** (300 → 255). `float32` has no such overflow, so do your maths there. The Ariane 5 rocket (1996) self-destructed from the same family of bug.



Figure 5. Left: the origin is the top-left; x counts columns, y counts rows. **Middle:** an OpenCV image (stored BGR) shown as RGB gives the "blue cat" bug. **Right:** your Q1 answer drawn: the red sawtooth is NumPy wrapping, so bright pixels suddenly become dark.

6. Colour and the Bayer filter

Each photosite measures only brightness, so a mosaic of R, G and B filters sits on top and each pixel sees one colour. **Demosaicing** estimates the two missing colours, so about two-thirds of every photo's colour values are guessed. Green cells are doubled because the eye is most sensitive to green (figures in Lesson 4).

7. Worked example: an edge in numbers

In the 4 × 6 grid above, dark (≈ 20) on the left meets bright (≈ 200) on the right. The jump is 21 → 200 (a difference of 179), while neighbours elsewhere differ by about 2. Sliding a difference detector across the image is **convolution** (Stage 2).

Formulas to remember

Formula 1.1 **Sampling**

$$I[m, n] = I(m\Delta,\ n\Delta)$$

where

$I(x, y)$	continuous brightness of the scene image at position (x, y)
$I[m, n]$	the stored pixel value at row m, column n
m, n	whole-number row and column indices: $0, 1, 2, \ldots$
Δ	spacing between neighbouring samples (the pixel pitch)

In words: pixel $[m, n]$ is the scene's brightness measured at the point $(m\Delta, n\Delta)$.

Formula 1.2 **Quantisation levels and 8-bit conversion**

$$L = 2^b, \qquad q = \text{round}(v \times 255)$$

where

b	number of bits used per value
L	number of distinct brightness levels the bits can represent
v	brightness as a real number from 0 (black) to 1 (white)
q	the stored 8-bit integer, from 0 to 255
round	round to the nearest whole number

Formula 1.3 Grayscale (luma) from colour

$$Y = 0.299\,R + 0.587\,G + 0.114\,B$$

where

Y	perceived brightness (luma), used as the grayscale value
R, G, B	the pixel's red, green and blue values
the weights	0.299, 0.587 and 0.114; they sum to 1, and green is weighted most because the eye is most sensitive to it

Formula 1.4 Image memory

$$\text{bytes} = H \times W \times C \times \text{bytes per value}$$

where

H	image height: number of rows
W	image width: number of columns
C	number of channels: 1 for grayscale, 3 for colour
bytes per value	1 for `uint8`, 4 for `float32`

Exercises, your answers and evaluation

Q1. Predict: a `uint8` pixel is 200. What is `img + 100` in NumPy?

YOUR ANSWER

44. 200 + 100 = 300, but `uint8` holds only 0–255 and NumPy wraps modulo 256, so 300 mod 256 = 44. The pixel gets darker.

EVALUATION

Correct, with exactly the right reasoning. OpenCV's `cv2.add` saturates to 255 instead. Wrapping corrupts only bright regions, so a naive `uint8` blur leaves dark spots in highlights, which makes the bug hard to notice.

a mod 256 means "the remainder after dividing a by 256".

Q2. Mission: hide a message (steganography) and write the decoder.



Figure 6. The two photos look identical (each value changes by at most 1 level), but the last-bit plane gives the secret away: the hidden bits form a regular texture in the top rows.

YOUR ANSWER

```
flat = cv2.imread("secret.png").flatten()
print(np.packbits(flat[:64] & 1).tobytes().decode())   # -> "Hello CV"
```

EVALUATION

Perfect; `& 1` is the cleanest way to extract the last bit. Real tools hide a **header** (the message length) or end with a **terminator** byte, the same framing problem as a WebSocket stream.

Q3. Why would saving `secret.png` as JPEG destroy the message?

YOUR ANSWER

JPEG is lossy: it rounds away fine detail in 8 × 8 blocks (DCT + quantisation), so values come back off by several levels. PNG is lossless.

EVALUATION

Exactly right. Even quality 100 is not fully lossless. Image forensics spots edits for the same reason: editing disturbs JPEG's block statistics.

Q4. Memory of a 1920 × 1080 colour frame in `float32`, and per second at 30 fps.

YOUR ANSWER

1920 × 1080 × 3 × 4 = 24,883,200 bytes ≈ 24.9 MB per frame, and ≈ 746 MB/s at 30 frames per second: four times the `uint8` size.

EVALUATION

Correct. Real pipelines keep frames in `uint8` as long as possible and often use 16-bit floats for deep learning.

Tips and tricks

- **Convert early, convert back late:** call `img.astype(np.float32)` after loading; clip and convert to `uint8` only to save or display.
- **Row first:** whenever you write `img[a, b]`, say "*y*, *x*" in your head.
- **Blue-skinned people (or blue cats)** mean an OpenCV BGR image shown with Matplotlib. Fix with `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`.
- **Sanity print:** `print(img.shape, img.dtype, img.min(), img.max())`.
- **Never do pixel-exact work on JPEGs.** Save working files as PNG.

LESSON 2

The pinhole camera and perspective projection

A pinhole lets each scene point reach the screen along exactly one straight line, so an image forms. Similar triangles then give $x = fX/Z$, the most important equation in classical computer vision.

Story to remember

Mozi (China, about 400 BCE) saw an upside-down image through a small hole in a dark room. Aristotle saw crescent suns under trees during an eclipse: each gap between leaves is a pinhole. Ibn al-Haytham (about 1021 CE) explained it with straight-line light. "Camera obscura" means "dark room", the origin of the word camera.

Key symbols in this lesson

f	focal length: distance from the pinhole to the image plane
X, Y, Z	a 3D point: X sideways, Y up/down, Z depth (distance in front of the camera)
x, y	where that point lands on the image plane, measured from the image centre
H	real height of an object (here *not* image height as in Lesson 1)
h	height of the object's image on the sensor

Core concepts

1. Why a hole makes an image

Without a hole, every point on the paper gets light from every scene point, so it all mixes into a glow. With a pinhole, each point on the paper receives light from only one direction, so one scene point maps to one image point. Rays cross at the hole, so the real image is **upside down**.

2. Similar triangles

The outside triangle (object height H, distance Z) and the inside triangle (image height h, focal length f) share the same angle at the pinhole, so $h/f = H/Z$. Computer vision puts a **virtual image plane** in front of the pinhole at distance f, which makes the image upright and removes the minus sign.

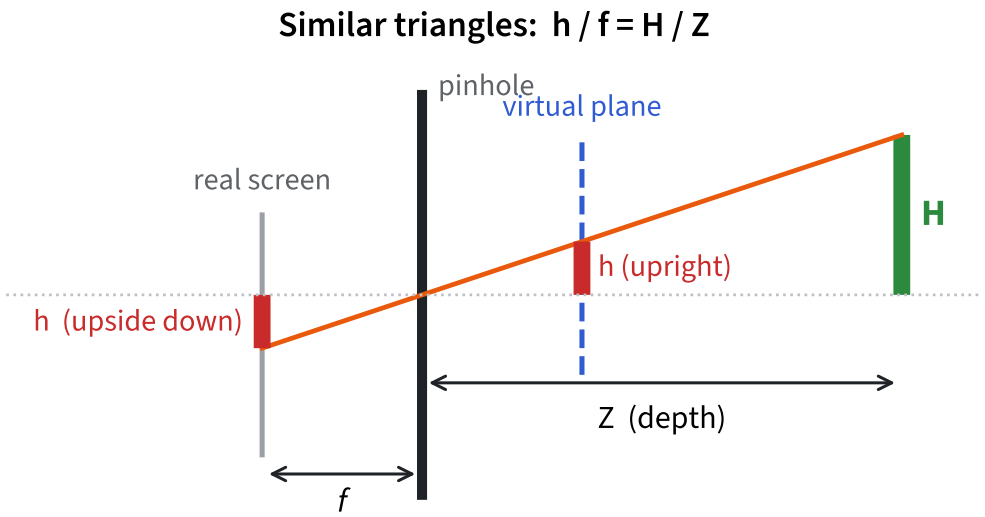


Figure 7. The orange ray makes two similar triangles: inside (f and h) and outside (Z and H). The real screen behind the pinhole shows h upside down; the virtual plane in front shows it upright.

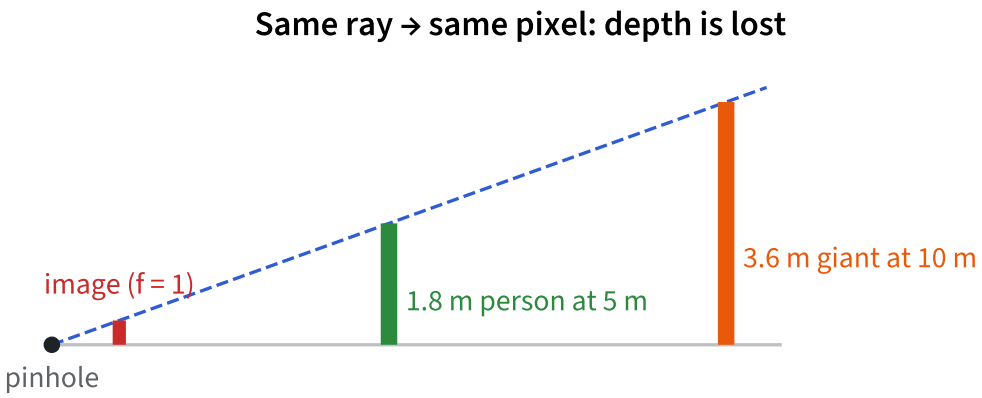


Figure 8. A 1.8 m person at 5 m and a 3.6 m giant at 10 m sit on the same blue ray, so they land on the same pixel: **depth is lost**.

- Bigger Z (walk away) → smaller image; bigger f (deeper box) → bigger image, which is what "zooming in" physically means.
- **Worked example:** a 1.8 m person at 5 m with $f = 40$ mm gives $h = 40 \times 1.8/5 =$ **14.4 mm**.

3. What dividing by Z does

- **Far things look small:** double the distance, half the image.
- **Parallel lines meet** at a **vanishing point** (Brunelleschi, about 1415).
- **Depth is lost:** forced-perspective Pisa photos, hobbits in *The Lord of the Rings* and the Ames room all exploit it. Recovering depth needs stereo, motion, known sizes or learned depth.

4. The pinhole dilemma

A big hole gives light but blur; a tiny hole gives sharpness but almost no light (and too tiny a hole causes diffraction). Early photos needed minutes to hours, so moving people vanished. The lens solves this (Lesson 3).

5. The dolly zoom (Vertigo, 1958; Jaws, 1975)

Move the camera back while zooming in. If f and Z grow together, the ratio f/Z stays the same, so the subject keeps its size; the background sits at a different depth, so it appears to rush forward.

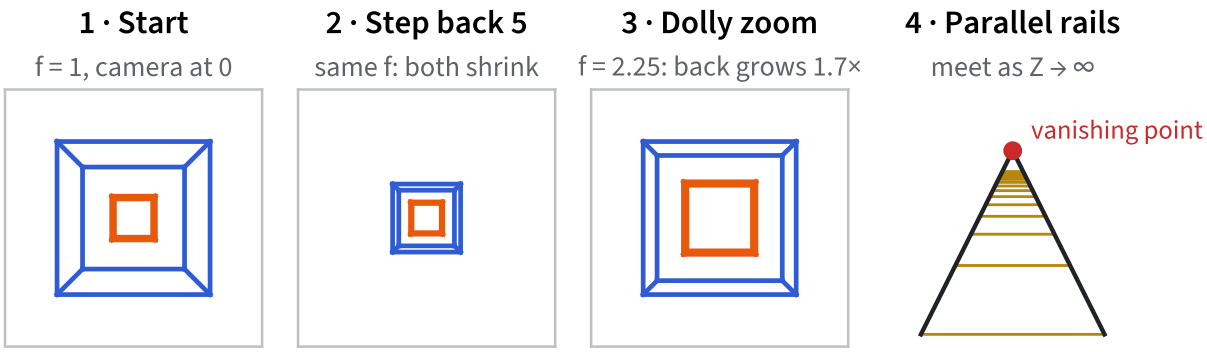


Figure 9. Blue: a front cube spanning $Z = 4$ to 6. Orange: a background cube spanning $Z = 14$ to 16. From panel 1 to panel 3 the front cube keeps its size while the background grows about 1.7×, exactly your Q1 prediction. Panel 2 shows that stepping back alone shrinks everything; panel 4 shows parallel rails meeting at a vanishing point.

Formulas to remember

Formula 2.1 Pinhole image height

$$\frac{h}{f} = \frac{H}{Z} \quad \Longrightarrow \quad h = \frac{f\,H}{Z}$$

where

H	real height of the object
Z	distance from the pinhole to the object (depth)
f	focal length: distance from the pinhole to the image plane
h	height of the object's image on the sensor

Units: H and Z must use the same unit; h comes out in the unit of f.

Formula 2.2 Perspective projection

$$x = \frac{f\,X}{Z}, \qquad y = \frac{f\,Y}{Z}$$

where

X, Y, Z	3D position of a scene point in camera coordinates (Z = depth along the viewing direction)
x, y	2D position of its image on the image plane, measured from the centre
f	focal length

In words: projection = "divide by depth".

Formula 2.3 Distance from a known size, and size in pixels

$$Z = \frac{f\,H}{h}, \qquad h_{\text{px}} = \frac{h_{\text{mm}}}{\text{pixel pitch (mm)}}$$

where

Z	distance to the object, recovered from its known real height H
h_{mm}	image height on the sensor in millimetres
pixel pitch	width of one pixel on the sensor in millimetres (1 µm = 0.001 mm)
h_{px}	image height counted in pixels

Mission: a virtual camera (3D cube renderer)

Pseudocode. Build the 8 cube corners (each coordinate ±1); keep the 12 edges whose corners differ in exactly one coordinate; add 5 to every Z; project with $x = fX/Z$, $y = fY/Z$; draw each edge.

```
def project(P, f):
    # P: N x 3 array of points (X, Y, Z); returns N x 2 image points (x, y)
    return f * P[:, :2] / P[:, 2:3]   # x = fX/Z, y = fY/Z, broadcast over rows
```

Result. A small square inside a bigger square (panel 1 of the dolly-zoom figure). The front face ($Z = 4$) is drawn larger than the back face ($Z = 6$); that difference is perspective.

Exercises, your answers and evaluation

Q1. Dolly zoom: add a background cube at $Z = 15$, move the camera back 5, and choose f so the front cube keeps its size. Predict the background.

YOUR ANSWER

The front face moves from $Z = 4$ to 9, so $f = 9/4 = 2.25$ (or $f = 2$ matching the cube's centre, $5 \to 10$). The background's front face goes from $1/14 \approx 0.071$ to $2.25/19 \approx 0.118$, about 1.7× bigger, rushing toward you: the Vertigo effect.

EVALUATION

Correct, with real insight. That $f = 2.25$ and $f = 2$ differ shows no single f keeps the whole front cube unchanged, because its faces sit at different depths. Stepping back makes it look flatter: **perspective compression**. A selfie at about 30 cm makes the nose look about 30% wider than a photo from 1.5 m; portrait photographers stand back with longer lenses.

Q2. Phone camera: $f \approx 5$ mm, 1 µm pixels, a 1.8 m friend at 10 m. Image height in mm and in pixels?

YOUR ANSWER

$h = 5 \times 1.8/10 = 0.9$ mm; $h_{px} = 0.9/0.001 = 900$ pixels.

EVALUATION

Exactly right. A pedestrian detector needing at least 50 pixels of height can only see so far.

Q3. Why are dark-restaurant photos blurry or grainy?

YOUR ANSWER

Not enough light: a longer shutter turns hand shake into blur; a higher ISO amplifies noise together with the weak signal, making it grainy.

EVALUATION

Perfect. Phones add night mode: average many short noisy frames; averaging K frames cuts noise by $\sqrt{K}$ (see the Lesson 4 noise figures).

Tips and tricks

- **Units trick:** in $h = fH/Z$, H and Z need the same unit; h comes out in f's unit.
- **One-line intuition:** "projection = divide by depth". Shrinking, vanishing points, lost depth and the dolly zoom all follow.
- **Draw it:** sketch the two triangles from memory; the formula falls out.
- **Real-world uses:** driver-assist range from $Z = fH/h$ (fails for an unusually tall truck), AR objects placed using the real camera's f, sports broadcast lines.
- **Calibration preview:** Stage 3 measures a real camera's f and image centre so these equations work in pixels.

LESSON 3

Lenses, focus and depth of field

A lens beats the pinhole dilemma by bending rays from one scene point back to one image point. But a flat sensor sits at only one focus distance, so only one distance is ever perfectly sharp.

Story to remember

In 1840 Joseph Petzval hand-calculated a lens (with military "human computers") that let in about 20× more light than earlier lenses: f/3.6 versus f/16 gives $(16/3.6)^2 \approx 20$. Portrait exposures fell from minutes to under a minute.

Key symbols in this lesson

f	focal length: where an object at infinity comes into focus behind the lens
Z	object distance: lens to object
z'	image distance: lens to the point where that object's sharp image forms
s	lens-to-sensor distance (what the focus ring sets)
Z_{focus}	the object distance that is currently in perfect focus
A	aperture diameter: width of the opening that lets light in
N	f-number, $N = f/A$ (written "f/N", e.g. f/2)
c	circle of confusion: diameter of the blur spot on the sensor

Core concepts

1. What a lens does

Glass slows light, so rays entering at an angle bend (**refraction**). A convex lens bends each ray just enough that rays from one point through the top, middle and bottom of the lens meet again at one point: a big opening and still a sharp image. Squinting works by turning your eye back into a pinhole.

2. Two special rays

The **central ray** passes straight through the lens centre, so $x = fX/Z$ still says where things land. A ray **parallel to the axis** bends through the focal point at distance f. Where they cross, the image forms.

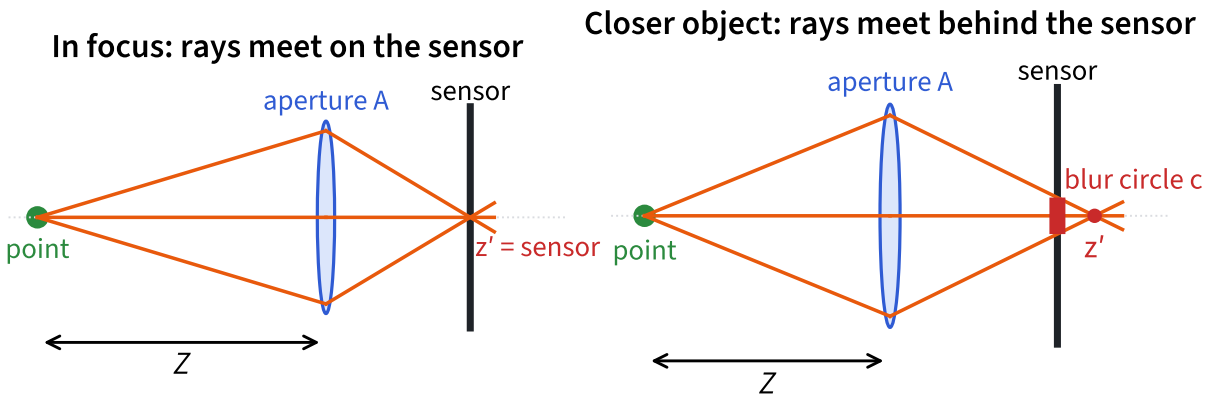


Figure 10. **Left:** the three orange rays meet exactly on the sensor, so the blur $c = 0$. **Right:** the object moved closer, so its rays meet at z' behind the sensor, and the sensor slices the cone of light as a red **blur circle** of diameter c.

3. Only one distance is sharp

With a 50 mm lens, each object distance Z focuses at a slightly different image distance z':

Table 4. Where a 50 mm lens forms a sharp image.

Object distance Z	Sharp image at z'
∞ (very far)	50.00 mm (= f)
5 m	50.51 mm
2 m	51.28 mm
0.5 m	55.56 mm

Far objects focus at exactly f: that is the true meaning of focal length. A focus ring moves the lens by fractions of a millimetre to choose which distance is sharp.

4. Circle of confusion and depth of field

The blur $c = 0$ at the focus distance and grows as the object moves away from it; c is **proportional to the aperture diameter** A. The range of distances that stays acceptably sharp ($c \leq$ about 0.03 mm on a full-frame sensor) is the **depth of field**.

5. The f-number

$N = f/A$, so f/2 on a 50 mm lens means a 25 mm opening. Each "stop" multiplies N by $\sqrt{2}$, which halves the opening's area and so halves the light.

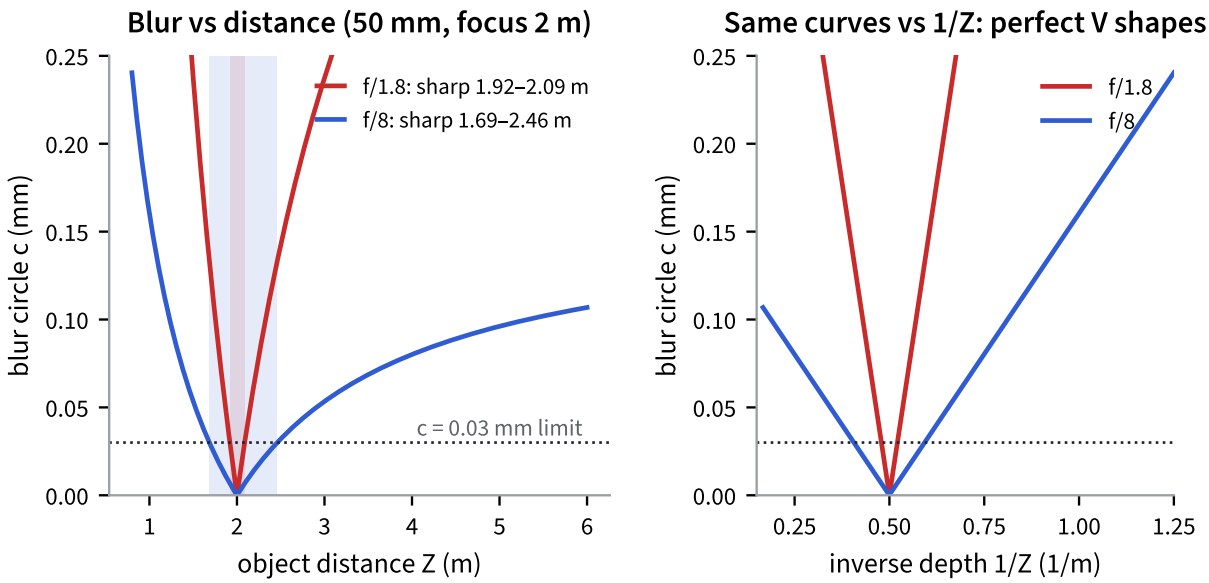


Figure 11. Blur c for a 50 mm lens focused at 2 m. **Left:** the shaded bands are the depth of field (your Q1 numbers). **Right:** plotted against inverse depth $1/Z$ the curves are perfect, symmetric V shapes; only in metres do they look lopsided.



Figure 12. Aperture sizes on the same lens. Each stop multiplies N by $\sqrt{2}$, halving the opening area and the light. f/16 behaves almost like a pinhole.

6. Real-world links

Phone **portrait mode** estimates a depth map (stereo, dual pixels or a neural net) and blurs in software; its failures (hair, glasses frames) are depth errors. **Autofocus** maximises a sharpness score such as the variance of the Laplacian, or uses phase detection (a tiny stereo system). Depth from defocus, focus stacking and factory lens design use the same formulas.

Formulas to remember

Formula 3.1 **Thin lens equation**

$$\frac{1}{f} = \frac{1}{Z} + \frac{1}{z'} \quad \Longrightarrow \quad z' = \frac{1}{1/f - 1/Z}$$

where

- f focal length of the lens
- Z object distance (lens to object)
- z' image distance (lens to where the sharp image forms)

Use one unit for all three, e.g. metres. As $Z \to \infty$, $1/Z \to 0$ and $z' \to f$.

Formula 3.2 **Circle of confusion (blur diameter)**

$$c = A\,\frac{|s - z'|}{z'} = A\,s\left|\frac{1}{Z_{\text{focus}}} - \frac{1}{Z}\right|$$

where

- c diameter of the blur spot on the sensor
- A aperture diameter
- s lens-to-sensor distance
- z' where this object would focus (from the thin lens equation)
- Z_{focus} the distance the lens is focused at
- Z the object's actual distance

The first form is similar triangles on the cone of light; the second follows by substituting the thin lens equation. Blur is linear in $1/Z$.

Formula 3.3 f-number, light and blur

$$N = \frac{f}{A}, \qquad \text{light} \propto \frac{1}{N^2}, \qquad c \approx \frac{f^2}{N}\left|\frac{1}{Z_{\text{focus}}} - \frac{1}{Z}\right|$$

where

N	f-number (the “8” in f/8)
f, A	focal length and aperture diameter
light	amount of light reaching the sensor; $\propto$ means “proportional to”
c	blur diameter (approximation uses $s \approx f$)

Formula 3.4 Focus travel and hyperfocal distance

$$\Delta z' \approx \frac{f^2}{Z}, \qquad H_{\text{hyp}} \approx \frac{f^2}{N\,c}$$

where

$\Delta z'$	how far the lens must move from infinity focus to focus on distance Z
H_{hyp}	hyperfocal distance: focus here and everything from $H_{\text{hyp}}/2$ to infinity is acceptably sharp
c	largest acceptable blur (about 0.03 mm on full frame)

Mission: map the depth of field

Pseudocode. Fix a 50 mm lens focused at 2 m and compute the sensor position s; for object distances 0.5–10 m compute z'; compute c for f/1.8 and f/8; plot with a line at 0.03 mm; read where each curve stays below it. The left panel of the depth-of-field figure is the expected result.

```
img_dist = lambda Z: 1 / (1/f - 1/Z)    # thin lens: z' for an object at Z
A = f / N                               # aperture diameter from the f-number
c = A * np.abs(s - img_dist(Z)) / img_dist(Z)   # blur circle diameter
```

Exercises, your answers and evaluation

Q1. Read the depth-of-field range for f/1.8 and f/8. By what factor is f/8 bigger, and why?

YOUR ANSWER

f/1.8: about 1.92–2.09 m (0.17 m); f/8: about 1.69–2.46 m (0.78 m); roughly 4.6×. Since $c = A\,s\,|1/Z_{\text{focus}} - 1/Z|$, blur $\propto A$. $A = 27.8$ mm versus 6.25 mm is 4.44× smaller, so objects can be 4.44× further off in $1/Z$. In metres it comes out slightly larger (4.6×) and stretches more behind the subject.

EVALUATION

Excellent. The sharp zone is **symmetric in** $1/Z$ (right panel). Inverse depth returns later: stereo disparity $\propto 1/Z$. Bonus: the **hyperfocal distance** $H_{\text{hyp}} \approx f^2/(Nc)$; for 50 mm at f/8, $H_{\text{hyp}} \approx 10.4$ m, and focusing there makes everything from about 5.2 m to infinity sharp.

Q2. How far must a 50 mm lens move between a mountain and a flower at 0.5 m? Why do phone lenses move far less?

YOUR ANSWER

z' goes from 50 to 55.56 mm, about 5.6 mm of travel. Phone ($f = 5$ mm): 5.000 → 5.051 mm, about 0.05 mm. Travel $\Delta z' \approx f^2/Z$, so a 10× smaller f means about 100× less travel.

EVALUATION

Correct, and f^2/Z is exactly the right scaling law.

Q3. Why does a phone's tiny lens keep almost everything sharp? Does f or N do the work?

YOUR ANSWER

With $A = f/N$ and $s \approx f$, $c \approx (f^2/N)\,|1/Z_{\text{focus}} - 1/Z|$. N is similar for both cameras, but f is squared, so a 10× shorter f gives about 100× less blur (about 10× less relative to the smaller frame). So f does the work, and phones fake background blur with depth estimation.

EVALUATION

Correct, including the stricter limit for small sensors. A phone at "f/1.8" behaves for depth of field like a full-frame camera at about f/12.

Tips and tricks

- **Think in** $1/Z$**:** if a depth-of-field result looks lopsided in metres, re-check it in $1/Z$.

- **Every stop halves the light** because N goes up by $\sqrt{2}$ and light goes as $1/N^2$.
- **Big f-number = small hole:** f/16 is near-pinhole (deep focus); f/1.4 is wide open (blurry background).
- **Focal length = where infinity focuses**, not the box depth from Lesson 2.
- **Small f wins** because it is squared in both blur and focus travel.

LESSON 4

Sensors: exposure, noise and colour

A sensor pixel is a photon bucket. Because photons arrive randomly, noise is physics (SNR = $\sqrt{\lambda}$), and the only real fixes are more photons or averaging frames.

Story to remember

Boyle and Smith sketched the CCD at Bell Labs in 1969 in about an hour (Nobel Prize 2009). Einstein's 1921 Nobel Prize was for the photoelectric effect, the physics every sensor runs on. Bryce Bayer patented the RGGB mosaic at Kodak in 1976.

Key symbols in this lesson

λ	average number of photons a pixel collects in one exposure
σ	standard deviation of that count: the size of the shot noise
SNR	signal-to-noise ratio: signal divided by noise (higher is cleaner)
E	exposure: how much light a pixel collects
L	scene luminance (brightness of the scene; not the "levels" of Lesson 1)
t	shutter time; N = f-number (Lesson 3)
K	number of frames averaged together
QE	quantum efficiency: fraction of photons that free an electron

Core concepts

1. The photon bucket

Photons arrive → some free electrons (quantum efficiency about 50–80%) → electrons fill the well → voltage → amplifier (ISO) → analogue-to-digital converter (ADC) → pixel value. A full well saturates (blown-out white skies); a nearly empty one is noisy. **Dynamic range** is the largest usable signal divided by the smallest; HDR merges short and long exposures.



Figure 13. A pixel is a photon bucket: each step from photons to a stored number.

2. Exposure and its costs

Exposure ∝ scene brightness × shutter time t × aperture area (and aperture area ∝ $1/N^2$). Every way of getting more light has a price:

Table 5. What doubling the light costs you.

Control	Doubling the light costs you
Shutter time t	Motion blur
Aperture (lower N)	Shallower depth of field
ISO	Adds no light; amplifies signal and noise equally

3. Rolling shutter

CMOS sensors read rows from top to bottom, so fast motion bends. Robots and factory cameras use a **global shutter** that captures all rows at once (Raspberry Pi sells one).

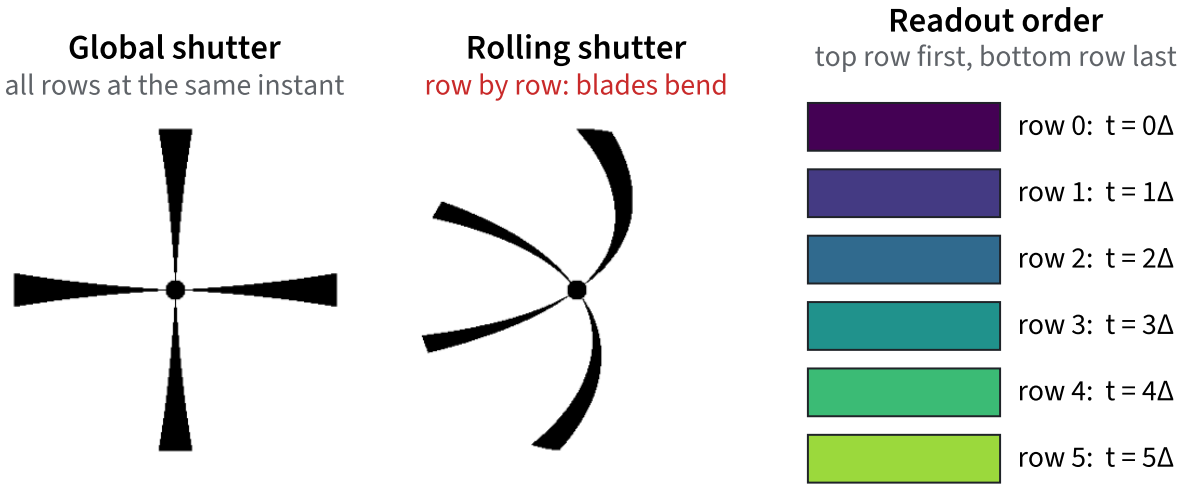


Figure 14. The same propeller captured all at once (global) and row by row (rolling). Each row sees the blades at a later moment t, so they bend and detach. Δ is the time between reading two rows.

4. Noise

- **Shot noise:** photon counts follow a Poisson distribution with mean λ and standard deviation $\sqrt{\lambda}$. It is unavoidable.
- **Read noise:** a constant electronic error added each time the sensor is read; it dominates in near-darkness.
- **Dark current:** heat frees electrons with no light at all; astronomy cameras are cooled.
- **Quantisation noise:** the rounding to whole levels from Lesson 1.

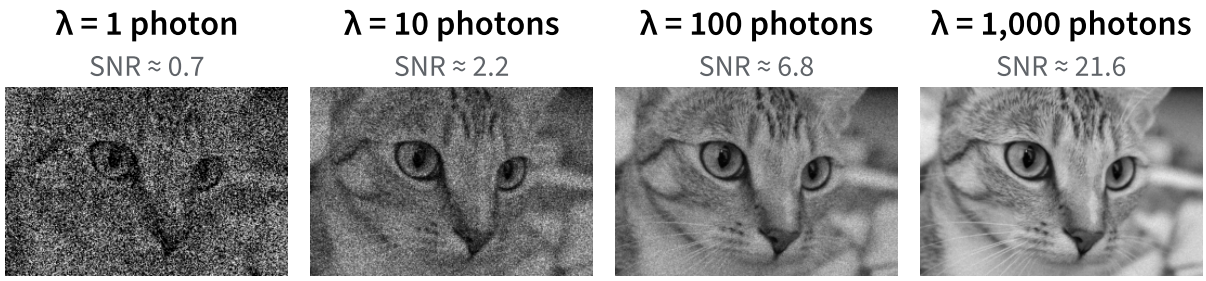


Figure 15. Grain fades as the photon count λ grows. SNR grows only as $\sqrt{\lambda}$.

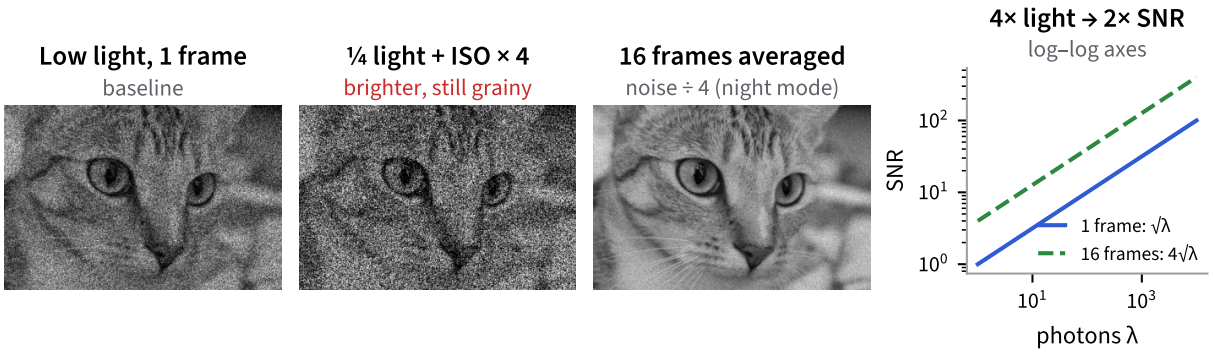


Figure 16. Raising ISO makes the frame brighter but just as grainy; averaging 16 frames cuts noise 4× (night mode). The log–log plot shows that 4× the light gives only 2× the SNR.

Table 6. Shot noise in numbers.

Photons λ	Noise $\sqrt{\lambda}$	SNR	Noise as % of signal
1	1	1	100%
100	10	10	10%
10,000	100	100	1%

5. Bayer mosaic and demosaicing

The repeating 2 × 2 tile is R G / G B, so half of all pixels are green. The raw output is a single $H \times W$ array with one colour value per pixel. Demosaicing fills in the missing colours; **bilinear** demosaicing averages the neighbours. Typical errors are **false colour** and **zippering**. Alternatives: X-Trans (a 6 × 6 pattern), Foveon (stacked layers), Hubble (separate filtered exposures).

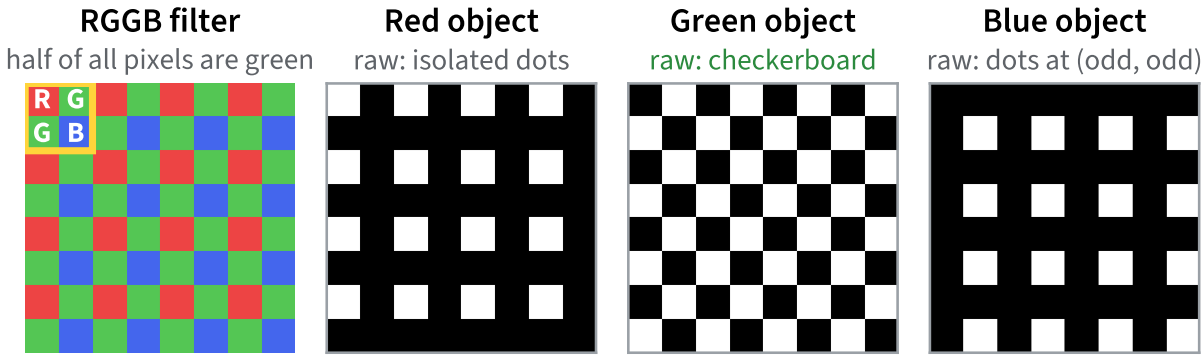


Figure 17. The correction to your Q1 answer. Red and blue each fill one cell in four, so a red or blue object shows isolated dots; only green fills two diagonal cells per tile, so only a green object shows a checkerboard.

6. The ISP pipeline

The camera's image signal processor (ISP) runs: 1 black level, 2 white balance, 3 demosaicing, 4 colour correction, 5 denoise and sharpen, 6 gamma, 7 JPEG. "The dress" (2015) went viral because brains disagreed on white balance.



Figure 18. The ISP step by step: a gray mosaic with one value per pixel, then dark colour with a cast, then natural colour after white balance (WB) and gamma, then visible JPEG blocks at low quality.

Formulas to remember

Formula 4.1 Exposure

$$E \propto L \cdot t \cdot \frac{1}{N^2}$$

where

E	exposure: light collected by a pixel
L	scene luminance (how bright the scene is)
t	shutter time
N	f-number; the light admitted falls as $1/N^2$
$\propto$	"is proportional to"

Formula 4.2 Shot noise and averaging

$$\sigma = \sqrt{\lambda}, \qquad \text{SNR} = \frac{\lambda}{\sqrt{\lambda}} = \sqrt{\lambda}, \qquad \text{SNR}_K = \sqrt{K}\ \text{SNR}_1$$

where

λ	mean number of photons collected per pixel
σ	standard deviation of the photon count (the noise)
SNR	signal ÷ noise = λ/σ
K	number of frames averaged
SNR_1, SNR_K	SNR of one frame, and of the average of K frames

Often written "$\sqrt{N}$ averaging"; K is used here so it is not confused with the f-number N.

Formula 4.3 Bilinear demosaicing (green at a red or blue pixel)

$$G(i,j) = \tfrac{1}{4}\big[\, G(i-1,j) + G(i+1,j) + G(i,j-1) + G(i,j+1) \,\big]$$

where

$G(i,j)$	estimated green value at row i, column j (a red or blue site)
$G(i \pm 1,j)$	the measured green neighbours above and below
$G(i,j \pm 1)$	the measured green neighbours left and right

Mission: build a sensor and a demosaicer

Pseudocode. Load RGB floats in 0–1 and trim to an even size; make an empty $H \times W$ raw array; copy R at (even, even), G at (even, odd) and (odd, even), B at (odd, odd); demosaic with the **superpixel** method (each 2×2 tile becomes one pixel); show original, mosaic and reconstruction.

```
raw[0::2, 0::2] = img[0::2, 0::2, 0]                    # R at (even, even)
raw[0::2, 1::2] = img[0::2, 1::2, 1]                    # G at (even, odd)
raw[1::2, 0::2] = img[1::2, 0::2, 1]                    # G at (odd, even)
raw[1::2, 1::2] = img[1::2, 1::2, 2]                    # B at (odd, odd)

demo = np.dstack([raw[0::2, 0::2],                          # R
                  (raw[0::2, 1::2] + raw[1::2, 0::2]) / 2,  # mean of the two Gs
                   raw[1::2, 1::2]])                        # B
```

Expected. The mosaic looks grayscale from afar with fine texture up close (ISP panel 1); the superpixel result has correct colour at half resolution.

Exercises, your answers and evaluation

Q1. Describe the raw mosaic on a bright red object, and how to get green at a red pixel at full resolution.

YOUR ANSWER

A checkerboard, because only red-filtered pixels capture strong red. Estimate green at a red pixel by averaging the four nearest greens.

EVALUATION

The interpolation is exactly right (bilinear demosaicing). The pattern, though, is a grid of **isolated dots**, not a checkerboard; the checkerboard appears on green (see the Bayer figure). Averaging neighbours with fixed weights is convolution: you invented it before Stage 2.

Q2. 400 photons: what is the SNR? After 2 stops less light? How many frames to recover? What does doubling ISO do?

YOUR ANSWER

$\sqrt{400} = 20$; 2 stops less is 4× less light, so 100 photons → SNR 10; averaging $K = 4$ frames brings it back to 20; doubling ISO does not improve SNR.

EVALUATION

All correct.

Q3. Why does a phone video of a spinning propeller show bent, detached blades?

YOUR ANSWER

Rows are captured one after another and the propeller moves in between, so blades look curved or detached.

EVALUATION

Correct. Blades moving with the readout direction stretch; blades moving against it get squashed, so blades on one propeller look different (see the rolling-shutter panel). Fix: a global shutter.

Tips and tricks

- **ISO is a volume knob, not a bigger bucket.** Only more photons improve SNR.
- **"4× the light, 2× the quality"** is $\sqrt{\lambda}$; **"16 frames, 4× less noise"** is $\sqrt{K}$.
- **Red = dots, green = checkerboard** in a raw mosaic.
- **Average-the-neighbours = convolution.**
- **Pipeline order matters:** white balance before demosaicing; gamma and JPEG last.

LESSON 5

Lens distortion and JPEG

The last two things that happen before you get your array are imperfect glass (radial distortion, fixed by backward mapping) and compression (JPEG deletes the high-frequency detail your eyes miss least).

Key symbols in this lesson

r	distance of a point from the image centre in the ideal (undistorted) image
r_d	distance where that point actually lands in the distorted photo
k_1, k_2	radial distortion coefficients of the lens
DCT	discrete cosine transform: rewrites an 8 × 8 block as 64 pattern weights
DC	the first DCT weight (top-left): the block's flat, average level
q	JPEG quality setting (1–100)
MSE, PSNR	mean squared error, and peak signal-to-noise ratio in decibels (dB)

Part A: Lens distortion

The hook

A door peephole or a GoPro video makes straight walls bulge. Pure perspective keeps straight 3D lines straight, so curved lines mean the lens bends light beyond the pinhole model.

Why it happens

Rays near the edge of a lens bend slightly too much or too little, and the error grows with distance from the centre: **radial distortion**.

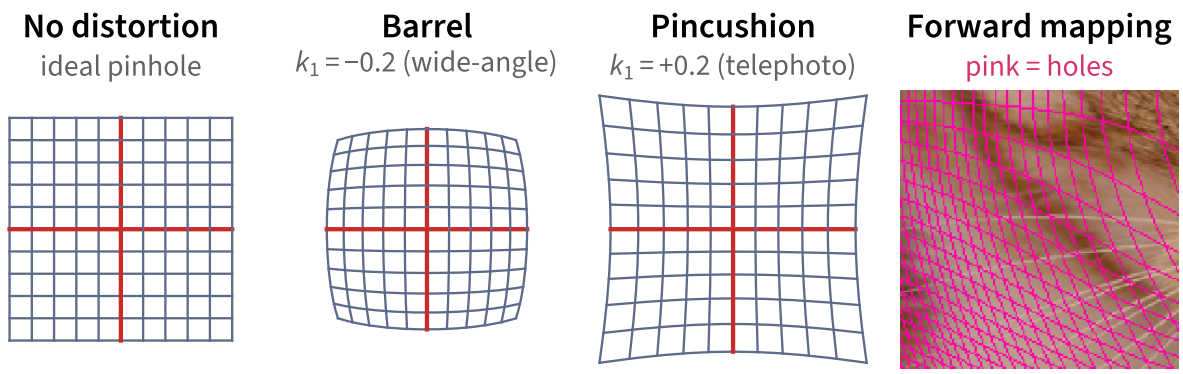


Figure 19. Barrel distortion bulges outward and pincushion pinches inward, while the red lines through the centre stay straight in both. The last panel is your Q3 answer made visible: forward mapping leaves **holes** (pink) where no source pixel landed.

Table 7. The two kinds of radial distortion.

Type	Edge magnification	Lines look	Typical lens	k_1
Barrel	Less at edges	Bulge outward	Wide-angle, fisheye, action and parking cameras	$k_1 < 0$
Pincushion	More at edges	Bow inward	Telephoto zooms	$k_1 > 0$

Worked example

With $k_1 = -0.2$ and $k_2 = 0$: a point at $r = 0.5$ lands at $r_d = 0.5 \times (1 - 0.2 \times 0.25) = 0.475$ (5% inward); a corner at $r = 1$ lands at 0.8 (20% inward). The centre barely moves.

Fixing it: backward mapping

For every *output* pixel, compute where it came from in the distorted image and interpolate that value. No holes, no collisions. k_1 and k_2 come from **camera calibration** with a checkerboard (`cv2.calibrateCamera`, Stage 3).

Where it matters

Car surround-view (four fisheyes undistorted and stitched into a top-down view), lane detection and measurement, and phone ultra-wides (which stretch people at the edges of group photos).

Part B: JPEG

The hook

A 12 MP photo is 36 MB raw but about 3 MB as JPEG (about 10× smaller) with no visible difference.

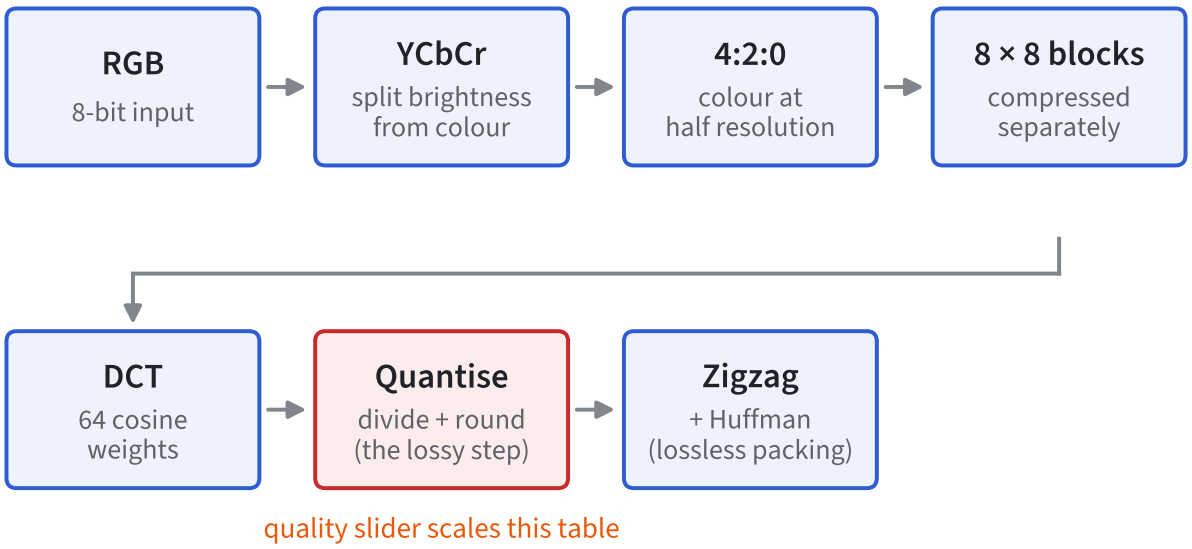


Figure 20. The JPEG encoder. Only the red quantise step throws information away; the quality slider scales its table.

1. **RGB → YCbCr:** separate brightness (Y) from colour (Cb, Cr); eyes see brightness detail far better.
2. **4:2:0 chroma subsampling:** store colour at half width and half height.
3. **8 × 8 blocks**, each compressed separately.
4. **DCT:** each block becomes a weighted mix of 64 cosine patterns, sorted from flat to fine.
5. **Quantise (lossy):** divide by a table and round; most fine coefficients become zero. The quality slider scales this table.
6. **Zigzag + Huffman:** pack the result losslessly.

Artefacts

Blocking (an 8 × 8 grid in smooth skies), **ringing** (ripples next to sharp edges) and **colour bleeding** (from half-resolution colour). Re-saving degrades further (“deep-fried” memes); forensics spots pasted regions by mismatched 8 × 8 grids; video calls go blocky when bandwidth drops.

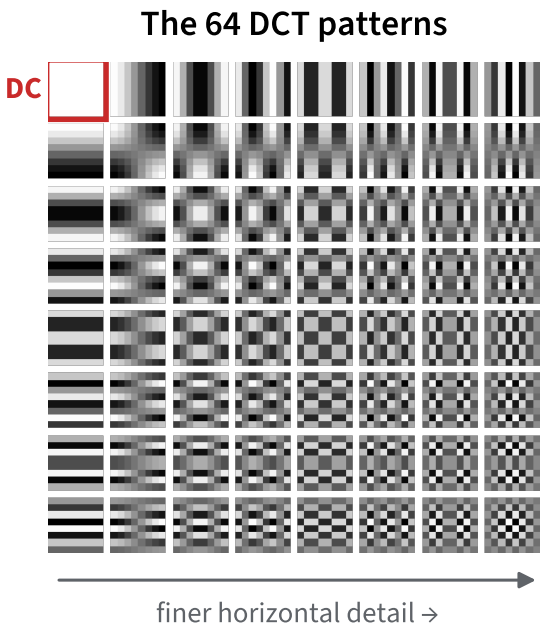


Figure 21. The 64 DCT patterns. Top-left is flat (DC); patterns get finer to the right and downward.

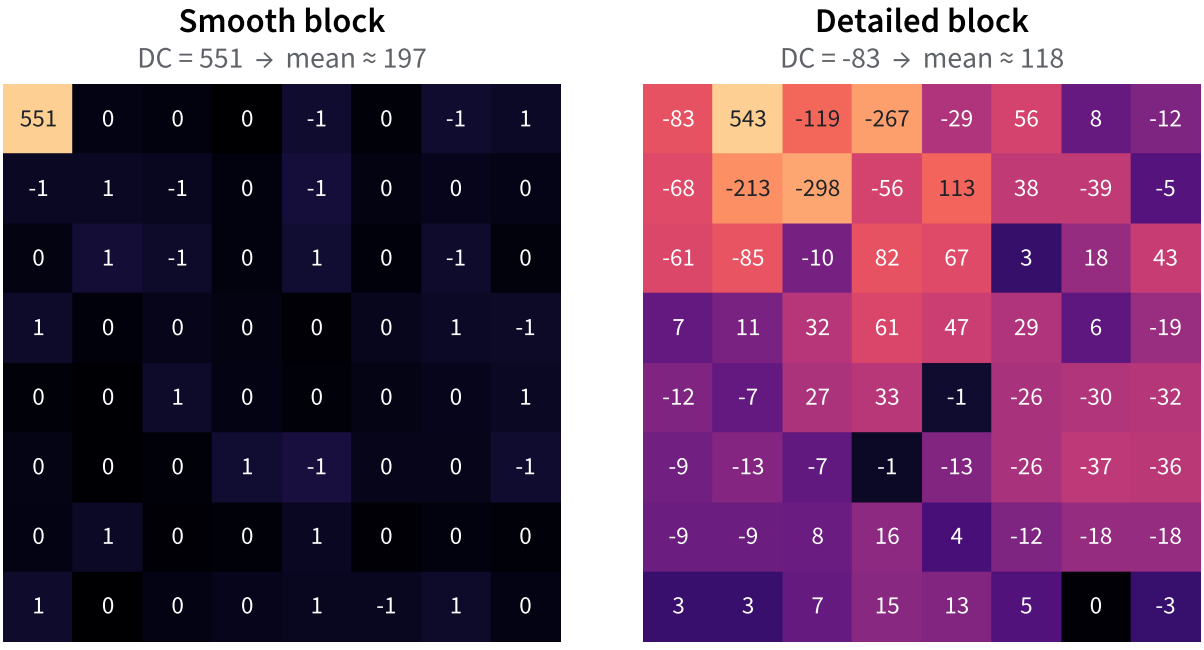


Figure 22. DCT weights of two 8 × 8 blocks (colour shows magnitude). The smooth block puts nearly everything in the top-left DC value, so zeroing the rest changes almost nothing; the detailed block spreads large values everywhere, which is why detail costs bits.

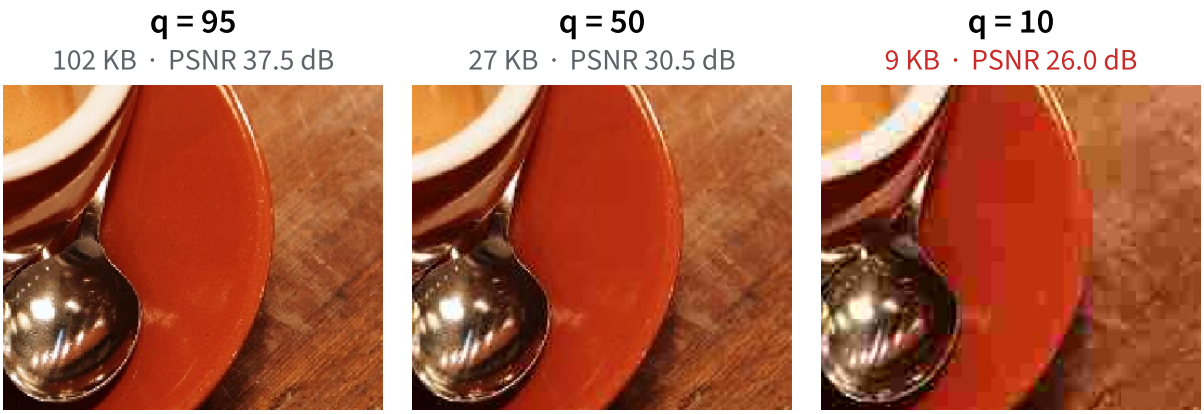


Figure 23. The same crop saved at three quality settings. At $q = 10$ the 8×8 blocks become clearly visible.

Formulas to remember

Formula 5.1 Radial distortion

$$r_d = r\left(1 + k_1 r^2 + k_2 r^4\right)$$

where

r	distance from the image centre where the point *should* be (normalised so the edge is about 1)
r_d	distance from the centre where it actually lands
k_1	main distortion coefficient: $k_1 < 0$ barrel, $k_1 > 0$ pincushion
k_2	smaller correction that matters mostly near the corners

Formula 5.2 Compression error: MSE and PSNR

$$\text{MSE} = \frac{1}{HW}\sum_{i,j}\left(I(i,j) - \hat{I}(i,j)\right)^2, \qquad \text{PSNR} = 10\log_{10}\frac{255^2}{\text{MSE}}$$

where

$I(i,j)$	original pixel value at row i, column j
$\hat{I}(i,j)$	the same pixel after compressing and decoding ("I-hat")
H, W	image height and width, so HW is the number of pixels
$\sum_{i,j}$	sum over every pixel
255	the largest possible 8-bit value
PSNR	error on a log scale in decibels (dB); higher means closer to the original

Rule of thumb: above 40 dB looks identical; about 35 dB is fine; below 30 dB the damage is clearly visible.

Formula 5.3 Block brightness from the DC coefficient

$$\text{mean brightness} = \frac{\text{DC}}{8} + 128$$

where

DC	top-left DCT weight from OpenCV's `cv2.dct` on a block shifted by −128
÷ 8	OpenCV's DCT makes DC equal to 8× the block mean
+ 128	undoes the −128 shift applied before the DCT

Mission: measure the cost of compression

Part 1. For q in 95, 50, 10, 2: encode to JPEG in memory, decode, compute MSE and PSNR, record the size, save as PNG.

Part 2. Take one 8 × 8 grayscale block, subtract 128, apply `cv2.dct`, print the 64 coefficients.

```
ok, buf = cv2.imencode(".jpg", img, [cv2.IMWRITE_JPEG_QUALITY, q])  # compress
 ↪  in memory
dec = cv2.imdecode(buf, cv2.IMREAD_COLOR)                          # decompress
mse = np.mean((img.astype(np.float32) - dec) ** 2)

block = gray[100:108, 100:108].astype(np.float32) - 128   # float first: no
 ↪  uint8 wrap
print(cv2.dct(block))                                    # the 64 pattern weights
```

Exercises, your answers and evaluation

Q1. Quality sweep: when do 8 × 8 blocks first appear, and why does size fall faster than PSNR from q = 95 to q = 50?

YOUR ANSWER

Quality q	Size (KB)	PSNR (dB)
95	481.9	47.7
50	125.8	35.1
10	55.7	29.3
2	39.9	23.4

JPEG removes high-frequency detail first, so size falls with little visual loss; at low quality 8 × 8 blocks become noticeable.

EVALUATION

Correct reading, but two parts were skipped. From q = 95 to 50 the file shrank 3.8× while PSNR fell 12.6 dB, yet 35 dB looks nearly identical: at q = 95 most bits store **sensor noise** and old artefacts, which are expensive and invisible. Blocks usually appear around q = 10–20 in smooth regions.

Q2. Smooth versus detailed DCT block: why does zeroing the bottom-right barely matter in smooth regions?

YOUR ANSWER

A large DC (532) with small other values (−6 to 9) means a smooth block; removing high frequencies changes little because they are near zero.

EVALUATION

Correct. $532/8 + 128 \approx 195$: a bright, smooth patch. Still to do: run a detailed block and compare (the DCT-blocks figure shows what to expect).

Q3. What two problems would forward mapping cause?

YOUR ANSWER

Holes and overlapping pixels; backward mapping avoids holes; bilinear interpolation handles fractional positions.

EVALUATION

Exactly right: holes and collisions (pink in the distortion figure). Bilinear blending of four neighbours is again a weighted sum of neighbours: convolution, Stage 2.

Tips and tricks

- **Sign aid:** k_1 negative = barrel (a barrel bulges out); k_1 positive = pincushion (pins pull the sides in).
- **The straight-line-through-the-centre test** is a free calibration check.
- **Always map backward:** ask “where did this output pixel come from?”, then interpolate.
- **JPEG in one sentence:** move to frequencies, round away the ones you won’t miss.
- **PSNR anchors:** 40+ dB invisible, about 35 dB fine, under 30 dB visible damage.
- **Answer every part of a question.**

Master formula sheet

Every Stage 1 formula in pipeline order. The last column points to the figure that shows it; every symbol is defined in the glossary that follows.

#	Formula	What it means	Figure
	LESSON 1 · IMAGES AS NUMBERS		
1	$I[m, n] = I(m\Delta,\ n\Delta)$	Sampling: continuous image → pixel grid	L1 sampling
2	$L = 2^b,\quad q = \text{round}(v \times 255)$	Quantisation levels; 8-bit conversion	L1 banding
3	$Y = 0.299R + 0.587G + 0.114B$	Grayscale (green weighted most)	L4 Bayer tile
4	bytes = $H \times W \times C \times$ bytes/value	Image memory	—
	LESSON 2 · PINHOLE CAMERA		
5	$h = fH/Z$	Pinhole image height	L2 similar triangles
6	$x = fX/Z,\quad y = fY/Z$	Perspective projection	L2 dolly zoom
7	$Z = fH/h$	Distance from a known object size	L2 lost depth
8	$h_{px} = h_{mm}/\text{pixel pitch}$	Image size in pixels	—
	LESSON 3 · LENSES		
9	$1/f = 1/Z + 1/z'$	Thin lens: where each distance focuses	L3 ray diagram
10	$c = A\,\lvert s - z'\rvert/z' = A\,s\,\lvert 1/Z_{\text{focus}} - 1/Z\rvert$	Circle of confusion; linear in $1/Z$	L3 V-curves

continued on next page

#	Formula	What it means	Figure
11	$N = f/A$, light $\propto 1/N^2$	Each stop ($\times\sqrt{2}$) halves the light	L3 aperture circles
12	$c \approx (f^2/N)\,\lvert 1/Z_{\text{focus}} - 1/Z \rvert$	Why small-f phones are sharp everywhere	—
13	$\Delta z' \approx f^2/Z$	Focus travel of the lens	—
14	$H_{\text{hyp}} \approx f^2/(Nc)$	Hyperfocal distance	—
LESSON 4 · SENSORS			
15	$E \propto L\,t/N^2$	Exposure	—
16	$\sigma = \sqrt{\lambda}$, SNR $= \sqrt{\lambda}$	Shot noise (Poisson)	L4 noise ladder
17	$\text{SNR}_K = \sqrt{K}\,\text{SNR}_1$	Averaging K frames	L4 averaging
18	$G = \frac{1}{4}(G_{\text{up}} + G_{\text{down}} + G_{\text{left}} + G_{\text{right}})$	Bilinear demosaicing	L4 Bayer
LESSON 5 · DISTORTION AND JPEG			
19	$r_d = r\,(1 + k_1 r^2 + k_2 r^4)$	Radial distortion ($k_1 < 0$ barrel)	L5 grids
20	PSNR $= 10 \log_{10}(255^2/\text{MSE})$	Compression error in dB	L5 quality ladder
21	mean = DC/8 + 128	Block brightness from OpenCV's DCT	L5 DCT blocks

Quick numbers worth memorising

- `uint8` 200 + 100 = 44 (NumPy) or 255 (OpenCV).
- A 1080p `float32` colour frame ≈ 24.9 MB.
- 1.8 m at 10 m with f = 5 mm and 1 µm pixels = 900 px.
- 50 mm lens travel to focus at 0.5 m ≈ 5.6 mm.
- 400 photons → SNR 20; 16 frames → 4× less noise.
- 50 mm at f/8: hyperfocal distance ≈ 10.4 m.

Notation glossary

Every symbol used in these notes, with the lesson where it appears. Letters that mean different things in different lessons are listed once per meaning.

Symbol	Meaning	Lesson
A	aperture diameter (width of the lens opening)	3
b	bits per stored value	1
B, G, R	blue, green and red channel values	1, 4
C	number of colour channels (1 or 3)	1
c	circle of confusion: blur-spot diameter on the sensor	3
DC	top-left DCT weight: a block's flat, average level	5
Δ	spacing between samples (pixel pitch)	1
$\Delta z'$	focus travel: how far the lens moves to refocus	3
E	exposure: light collected per pixel	4
f	focal length	2, 3
H	image height in rows	1, 5
H	real height of an object	2
H_{hyp}	hyperfocal distance	3
$h, h_{\text{mm}}, h_{\text{px}}$	image height of an object; in mm; in pixels	2
$I(x, y)$	continuous brightness at position (x, y)	1
$I[m, n], I(i, j)$	stored pixel value at a given row and column	1, 5
$\hat{I}(i, j)$	pixel value after compression and decoding	5
i, j	row and column index	4, 5
K	number of frames averaged	4
k_1, k_2	radial distortion coefficients	5
L	number of brightness levels, 2^b	1
L	scene luminance (scene brightness)	4

continued on next page

Symbol	Meaning	Lesson
λ	mean photon count per pixel	4
m, n	row and column index of a pixel	1
MSE	mean squared error between two images	5
N	f-number, f/A	3, 4
PSNR	peak signal-to-noise ratio, in decibels (dB)	5
q	stored 8-bit value (0–255)	1
q	JPEG quality setting (1–100)	5
r, r_d	ideal and distorted distance from the image centre	5
s	lens-to-sensor distance	3
σ	standard deviation of the photon count (shot noise)	4
SNR	signal-to-noise ratio	4
t	shutter (exposure) time	4
v	brightness as a real number from 0 to 1	1
W	image width in columns	1, 5
X, Y, Z	3D position of a scene point; Z is depth	2
x, y	image-plane position (Lesson 2) or column and row (Lesson 1)	1, 2
Y	luma (grayscale brightness)	1
Z	object distance from the pinhole or lens	2, 3
Z_{focus}	distance the lens is focused at	3
z'	image distance: where an object's sharp image forms	3
$\propto$	"is proportional to"	all
$\approx$	"is approximately equal to"	all

Tips, tricks and common mistakes

Five ideas tie Stage 1 together. If you can explain each one while pointing at its figure, the stage is solid.

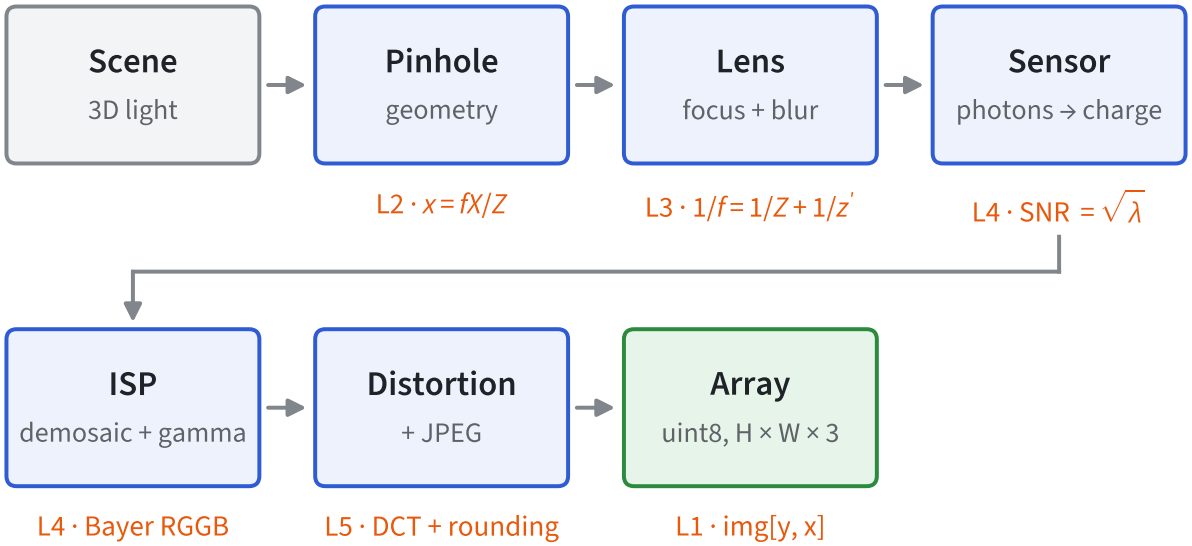


Figure 24. The Stage 1 pipeline again: each box is a lesson.

The five big threads

1. **Divide by depth.** $x = fX/Z$ explains perspective, vanishing points, lost depth, the dolly zoom and selfie noses.
2. **Think in $1/Z$.** Blur, focus and (later) stereo disparity are all linear in inverse depth.
3. **Square roots everywhere in noise.** SNR = $\sqrt{\lambda}$ and averaging K frames gives $\sqrt{K}$; ISO is just gain.
4. **Weighted sum of neighbours.** Demosaicing, bilinear interpolation and blurring are all convolution, the start of Stage 2.
5. **The image is a processed estimate.** Demosaicing, white balance, gamma and JPEG all change the numbers before `cv2.imread` returns them.

Common mistakes to avoid

Mistake	Symptom	Fix
Maths in `uint8`	Dark spots in bright areas (wrap-around)	`astype(np.float32)` first; clip and convert back last
BGR shown as RGB	Blue-skinned people or blue cats	`cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`
`img[x, y]`	Transposed or out-of-range crops	Always `img[row, col]` = `img[y, x]`
Pixel-exact work on JPEG	Hidden bits or measurements corrupted	Save working files as PNG
Raising ISO to “fix” noise	Brighter but equally grainy image	More light, a longer exposure, or averaging frames
Forward mapping a warp	Holes and collisions	Map backward, then interpolate
Trusting the f-number for blur across cameras	Phone “f/1.8” expected to blur like a DSLR	Blur scales with f^2/N and sensor size
Skipping sub-questions	Partial marks	Reread the prompt; answer every part

Ways to make it stick

- **Picture first:** cover the text, look at each figure and explain it aloud in 30 seconds. If you can’t, reread that concept.
- **Predict, then verify:** write the expected output and your reason before running any snippet.
- **Derive, don’t memorise:** re-derive $h = fH/Z$ and $c = A\,|s - z'|/z'$ from the triangle figures once a week.
- **Spot it in the wild:** moiré on a screen photo, crescent suns under trees, curved propellers, blocky video calls, stretched people at photo edges.
- **Sketch from memory:** the RGGB tile and the thin-lens ray diagram.
- **Explain it to a friend:** “Why is my restaurant photo grainy?” and “Why can’t one photo tell you how far away something is?”